

CS3524 Distributed Systems

A Replicated Key-Value Store using RAFT

Sam Kierdelewicz - 52320593

Najib Zarroug - 52320412

Table of Contents

CS3524 DISTRIBUTED SYSTEMS	1
1. INTRODUCTION	3
2. CONSENSUS AND RAFT BACKGROUND	3
3. SYSTEM ARCHITECTURE AND DESIGN DECISIONS	3
3.1 DESIGN DECISIONS BY PART	4
4. LEADER ELECTION	5
4.1 VOTE HANDLING	5
4.2 BECOMING LEADER	5
5. LOG REPLICATION, COMMITMENT, AND STATE MACHINE APPLICATION	5
5.1 CONSISTENCY CHECK ON THE FOLLOWER	6
5.2 BACKTRACKING VIA NEXT_INDEX AND MATCH_INDEX	6
5.3 COMMITMENT	6
6. CLIENT REQUEST HANDLING AND DEDUPLICATION	7
7. PERSISTENCE, SNAPSHOTTING, AND RECOVERY	8
7.1 SNAPSHOTTING	8
8. BEHAVIOUR UNDER FAILURES	8
8.1 DELAYED MESSAGES AND OVERLAPPING ELECTIONS	8
8.2 MESSAGE LOSS	8
8.3 NETWORK PARTITIONS	9
8.4 HEARTBEAT-CONFIRMED (READINDEX-STYLE) READS	9
9. EXTENSION: PRE-VOTE AND ROBUST CLIENT HANDLING	9
9.1 INTRODUCTION AND MOTIVATION	9
9.2 PROBLEM SCENARIO: PARTITION AND TERM INFLATION	10
9.3 DESIGN AND IMPLEMENTATION OF PRE-VOTE	11
9.4 SUPPORTING ROBUSTNESS FEATURES	11
9.4.1 Duplicate client request handling	12
9.4.2 Follower forwarding and stale-leader handling	12
10 TESTING AND EVALUATION	13
11. LIMITATIONS AND TRADE-OFFS	13
CONCLUSION	14
REFERENCES	15

1. Introduction

This report describes a distributed key-value store built on top of the Raft consensus algorithm. The cluster has five nodes (node-0 through node-4) connected through a central network process that simulates message routing under a range of conditions. Clients issue PUT, GET and DELETE operations through the network, and the cluster is expected to keep the visible state consistent across nodes even when messages are delayed, lost, reordered, or partitioned.

The implementation was developed in stages following the coursework brief. Parts 1 and 2 covered core leader election, log replication, and basic client request handling. Part 3 added robustness mechanisms: persistent state, snapshotting, request deduplication, and recovery. Part 4 introduced ReadIndex-style heartbeat-confirmed reads, so that GET operations are not served immediately from a leader that may have lost contact with the majority. Part 5 is the extension and adds the Pre-Vote phase from Ongaro's PhD thesis (Ongaro, 2014; Ongaro and Ousterhout, 2014) alongside supporting client-handling features that improve availability under partitions.

The Raft logic lives in `node.py`, which started as a skeleton with stub methods for election, vote handling, replication, client requests, persistence and snapshotting. The provided code in `node.py` handled network connection, message dispatch and the timer loop. The cluster code interacts with the network through `message.py`, in which the constructors for the Raft RPCs (`make_request_vote`, `make_vote_response`, `make_append_entries`, `make_append_entries_response`) were added on top of the supplied wire protocol, along with the `pre_vote` field for the extension and the `read_id` field used by heartbeat-confirmed reads. The network process and its failure-mode variants (`network.py`, `network_delayed.py`, `network_lossy.py`, `network_partition.py`, `network_chaos.py`), the supplied client (`client_interactive.py`, `client_test.py`), the run helper (`run_cluster.py`) and `config.py` were left as supplied, so the cluster runs unchanged against the provided test harnesses. A `test_extension.py` file was added for the Pre-Vote extension.

2. Consensus and Raft Background

Consensus is the problem of getting a set of independent processes to agree on a single ordered sequence of operations even when some processes can fail or messages can be lost (Tanenbaum and Van Steen, 2007; Lamport, 1998). In a replicated key-value store, agreement on operation order ensures that replicas apply the same committed operations in the same order. Read safety is then handled by serving GET requests through the leader using a heartbeat-confirmed quorum check, so that a leader which can no longer reach a majority does not return values that may have been overwritten elsewhere.

The classical FLP result shows that no fully asynchronous deterministic protocol can guarantee both safety and liveness with a single faulty process (Fischer, Lynch and Paterson, 1985). Real systems get around this by relying on partial synchrony, that is, on bounded message delays and timeouts that eventually fire reliably. Raft sits in this model. Raft preserves safety by ensuring that committed log entries are not overwritten and that future leaders contain all previously committed entries. It provides liveness whenever a majority of nodes can communicate within a few election timeouts.

Raft achieves this through three mechanisms that are reflected directly in the implementation: **leader election**, which ensures a single coordinator at any time; **log replication**, in which the leader copies operations to followers; and **majority-based commitment**, which is the rule that an entry only counts as durably accepted once it has been stored on more than half of the nodes (Ongaro and Ousterhout, 2014).

3. System Architecture and Design Decisions

The cluster uses a star topology. All five nodes connect through a single central process implemented in `network.py`, which forwards each message based on its `dst` field. A separate set of subclasses (`network_delayed.py`, `network_lossy.py`, `network_partition.py`, `network_chaos.py`) override the message-delivery method to simulate delays, drops and partitions. This design lets the test harness swap network conditions without changing any node code.

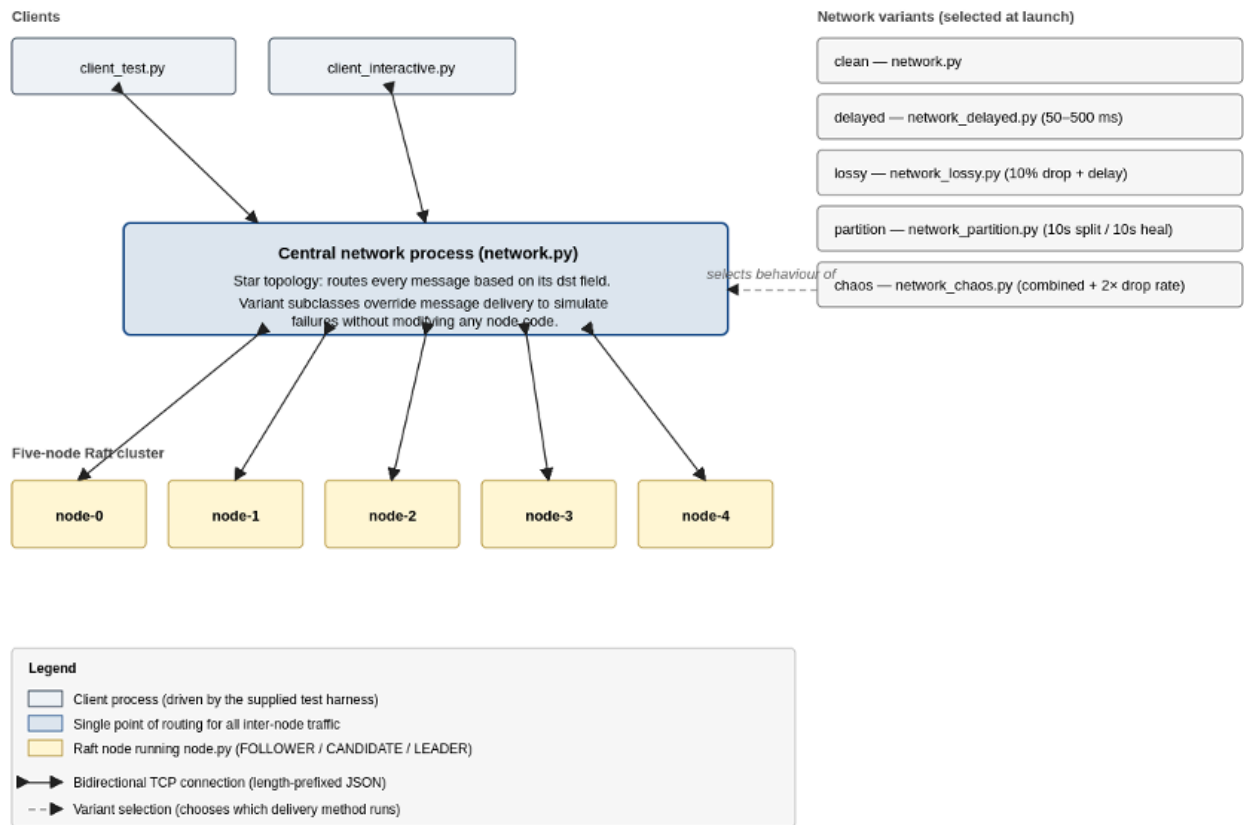


Diagram 1. Cluster topology and test-network variants. All inter-node traffic is routed through a single central network process whose delivery method is overridden by `network_delayed.py`, `network_lossy.py`, `network_partition.py` and `network_chaos.py` to simulate different conditions. The five Raft nodes and the client connect to the same central process, which lets the test harness swap network behaviour without modifying any node code.

The wire protocol is length-prefixed JSON over TCP, defined in `message.py`. Each Raft message carries the standard fields (`src`, `dst`, `type`, `term`) plus the fields specific to its type (vote requests carry `last_log_index` and `last_log_term`; `AppendEntries` carries `prev_log_index`, `prev_log_term`, `entries`, `leader_commit`, and an optional `read_id` used for confirmed reads). Pre-Vote re-uses the existing `REQUEST_VOTE` message type with an added `pre_vote` boolean rather than introducing a new RPC.

Clients issue three operations. `PUT(key, value)` and `DELETE(key)` modify state through the log; `GET(key)` is served only after the leader has confirmed it still holds quorum. Each request carries a (`client_id`, `request_id`) pair that the cluster uses to detect retries.

3.1 Design decisions by part

Part	Main design decision	Reason
1 / 2	Single-leader replication; commit only entries from the current term; randomised election timeouts in [1.5, 3.0] s.	Matches the original Raft paper (Ongaro and Ousterhout, 2014). The current-term commit rule prevents the well-known case where a previous-term entry could be undone by a later election.
3	Atomic file persistence using a tmp file plus <code>os.replace</code> ; per-request deduplication keyed by (<code>client_id</code> , <code>request_id</code>); snapshotting that truncates the log past <code>last_applied</code> .	Crash safety requires durable term, vote and log state. Atomic replace avoids torn writes. Deduplication prevents non-idempotent operations from being applied twice on retry.
4	Heartbeat-confirmed (ReadIndex-style) reads: <code>GET</code> only completes after the leader	A leader that has lost contact with the majority must not return a value that may have been

Part	Main design decision	Reason
	collects acknowledgements from a majority for a tagged heartbeat round.	overwritten by a new leader on the other side of a partition.
5	Pre-Vote probes before incrementing <code>current_term</code> ; unacked-response retries; follower forwarding of client requests; abandonment of pending reads on stale leaders.	Pre-Vote prevents term inflation by isolated nodes, which protects the standing leader across partitions. The remaining features keep client-visible behaviour sensible under loss and partition.

The main trade-off is consistency over availability. During a minority partition the cluster cannot safely commit new entries and reads cannot be confirmed; the implementation surfaces this to the client as a timeout rather than returning a possibly stale value. This follows the Raft paper's design and matches what the assignment brief describes as the correctness priority.

4. Leader Election

Each node is in one of three roles: FOLLOWER, CANDIDATE or LEADER. A follower expects to hear from a leader at least once per heartbeat interval (0.5 s in `config.py`). When a follower has not heard from a leader within its randomised election timeout (drawn from [1.5, 3.0] s), the timer thread invokes `start_election()`.

In this implementation, `start_election()` does not directly increment the term. Instead it begins the Pre-Vote probe described in Section 9. For the discussion in this section, the relevant point is that once a majority of pre-votes has been collected in `handle_request_vote_response()`, the node performs the standard Raft transition: it increments `current_term`, sets role to CANDIDATE, sets `voted_for` to its own id, persists the new state via `save_state()`, and broadcasts RequestVote messages to every peer. The vote count is tracked in `votes_received`, which starts as a set containing only the candidate itself.

4.1 Vote handling

Vote requests are processed in `handle_request_vote()`. Treating the real-vote path (the `pre_vote=False` branch in the code) the rules are:

- Stale-term rejection. If the candidate's term is strictly less than the receiver's `current_term`, the request is rejected and the receiver replies with its own (higher) term.
- Step down on higher term. If the candidate's term is strictly greater, the receiver calls `_step_down(new_term)`, which clears `voted_for`, becomes a follower, and persists the new term.
- One vote per term. The receiver may only grant a vote if `voted_for` is None or already matches the candidate. This prevents two nodes from winning in the same term.
- Log freshness. The candidate's log must be at least as up-to-date as the receiver's, evaluated lexicographically on (`last_log_term`, `last_log_index`). This is the leader-completeness check from the paper (Ongaro and Ousterhout, 2014).

If all four checks pass, the receiver records `voted_for`, persists the change with `save_state()`, resets its election timer, and replies with `vote_granted=true`. Otherwise it replies with `vote_granted=false`.

4.2 Becoming leader

Vote responses are handled in `handle_request_vote_response()`. The candidate counts only responses from its own `current_term` and ignores anything older. As soon as `votes_received` reaches a majority of `CLUSTER_SIZE` (three out of five), the node transitions to leader: it sets role to LEADER, records itself as `leader_id`, initialises `next_index` for every peer to `last_log_index + 1`, initialises `match_index` to zero, and immediately calls `send_heartbeats()` so followers learn the identity of the new leader before their own timers fire. The randomised election timeout is reset at the same time, which avoids spurious re-election right after winning.

5. Log Replication, Commitment, and State Machine Application

Replication is driven entirely by the leader. `send_heartbeats()` runs once per `HEARTBEAT_INTERVAL` and sends an AppendEntries message to every follower. The message carries `prev_log_index` (one less than `next_index` for that

follower), `prev_log_term` (the term of the entry at `prev_log_index`), any entries in the leader's log from `next_index` onward, and the current `commit_index`. Heartbeats and entry-bearing `AppendEntries` are the same message; an empty entries list is simply a heartbeat.

AppendEntries consistency check and backtracking

`prev_log_index` and `prev_log_term` let the follower verify the previous entry. On mismatch, `next_index` moves back using the `match_index` hint.

Leader log (term shown in each cell)

1 (T1)	2 (T1)	3 (T2)	4 (T2)	5 (T3)	6 (T3)
--------	--------	--------	--------	--------	--------

Follower log (stale, with conflicting entry at index 3)

1 (T1)	2 (T1)	3 (T1)	missing	missing	missing
--------	--------	--------	---------	---------	---------

After replication completes

1 (T1)	2 (T1)	3 (T2)	4 (T2)	5 (T3)	6 (T3)
--------	--------	--------	--------	--------	--------

Once an entry is replicated to a majority, the leader updates `commit_index` and `apply` advances `last_applied`, applying PUT/DELETE operations to `kv_store` in index order.

Round 1: leader sends `AppendEntries` with `prev_log_index` = 5

Follower has no entry at index 5, so the consistency check fails.
The follower replies with `success=false` and `match_index=2` (its last good index).
Leader uses the hint: `next_index[follower] = max(1, min(match_index+1, current-1))`.

Round 2: leader sends `AppendEntries` with `prev_log_index` = 2

`prev_log_term` = T1 matches the follower's index 2. Consistency check passes.
The follower truncates the conflicting suffix at index 3 and appends the leader's entries 3 (T2), 4 (T2), 5 (T3), 6 (T3). `save_state()` persists the new log.

Commitment rule

`handle_append_entries_response()` only directly commits entries whose term equals `current_term`.
Older-term entries are committed indirectly when a current-term entry above them is committed.
This avoids the well-known case where committing a previous-term entry on majority replication alone could be undone by a later election. `apply_committed()` applies entries in order to `kv_store`.

Diagram 2. `AppendEntries` consistency check and backtracking. The leader uses `prev_log_index` and `prev_log_term` to verify that the follower has the expected previous entry. On a mismatch the follower replies with a hint, and `next_index` is moved back so that the leader retries from a position that does match.

5.1 Consistency check on the follower

`handle_append_entries()` rejects the message immediately if the leader's term is stale. If the leader's term is strictly higher, the follower steps down before continuing. The follower then accepts the leader by setting `leader_id` and resetting the election timer.

Next, the consistency check: if `prev_log_index` is greater than the follower's last log index, or if the term of the follower's entry at `prev_log_index` does not match `prev_log_term`, the follower rejects the `AppendEntries` with a `match_index` hint set to its own last log index. Otherwise the follower processes each incoming entry: existing entries with the same index are kept if their term matches, but if a conflict is found the follower truncates everything from that index onward and appends the leader's entries. This matches the Raft paper's rule that the follower's log is overwritten only on conflict, never silently. Any change to the log calls `save_state()` so the new entries are durable.

5.2 Backtracking via `next_index` and `match_index`

When a follower rejects an `AppendEntries`, the leader uses the `match_index` hint included in the response to advance `next_index` for that follower. The exact rule in `handle_append_entries_response()` is `next_index[follower] = max(1, min(match_index + 1, current - 1))`, which guarantees forward progress (the index always decreases by at least one) but skips back closer to the actual matching prefix when the hint is informative. This is faster than the textbook "decrement by one" approach and avoids excessive round-trips after a long absence.

5.3 Commitment

On a successful `AppendEntries` response, the leader updates `match_index` and `next_index` for that follower and then attempts to advance `commit_index`. For each index above the current `commit_index`, the leader counts itself plus every follower whose `match_index` is at least that index. If the count reaches a majority and the entry's term equals `current_term`, `commit_index` is advanced. The current-term check is important: directly committing a previous-term entry purely on majority replication would re-introduce the figure-8 anomaly described by Ongaro and Ousterhout (Ongaro and Ousterhout, 2014). Older entries are committed indirectly when a current-term entry above them is committed, which is safe because once any current-term entry is committed, log matching forces the older entries to be present on a majority too.

After `commit_index` moves, `apply_committed()` walks `last_applied` up to `commit_index` and applies each entry to the `kv_store` in order: PUT writes the value, DELETE removes the key. GET is not committed as a log entry; it is handled separately through the read mechanism described in Section 8. When a committed entry corresponds to a client request, the leader sends the cached response and stores it in `unacked_responses` for retransmission if the response is lost.

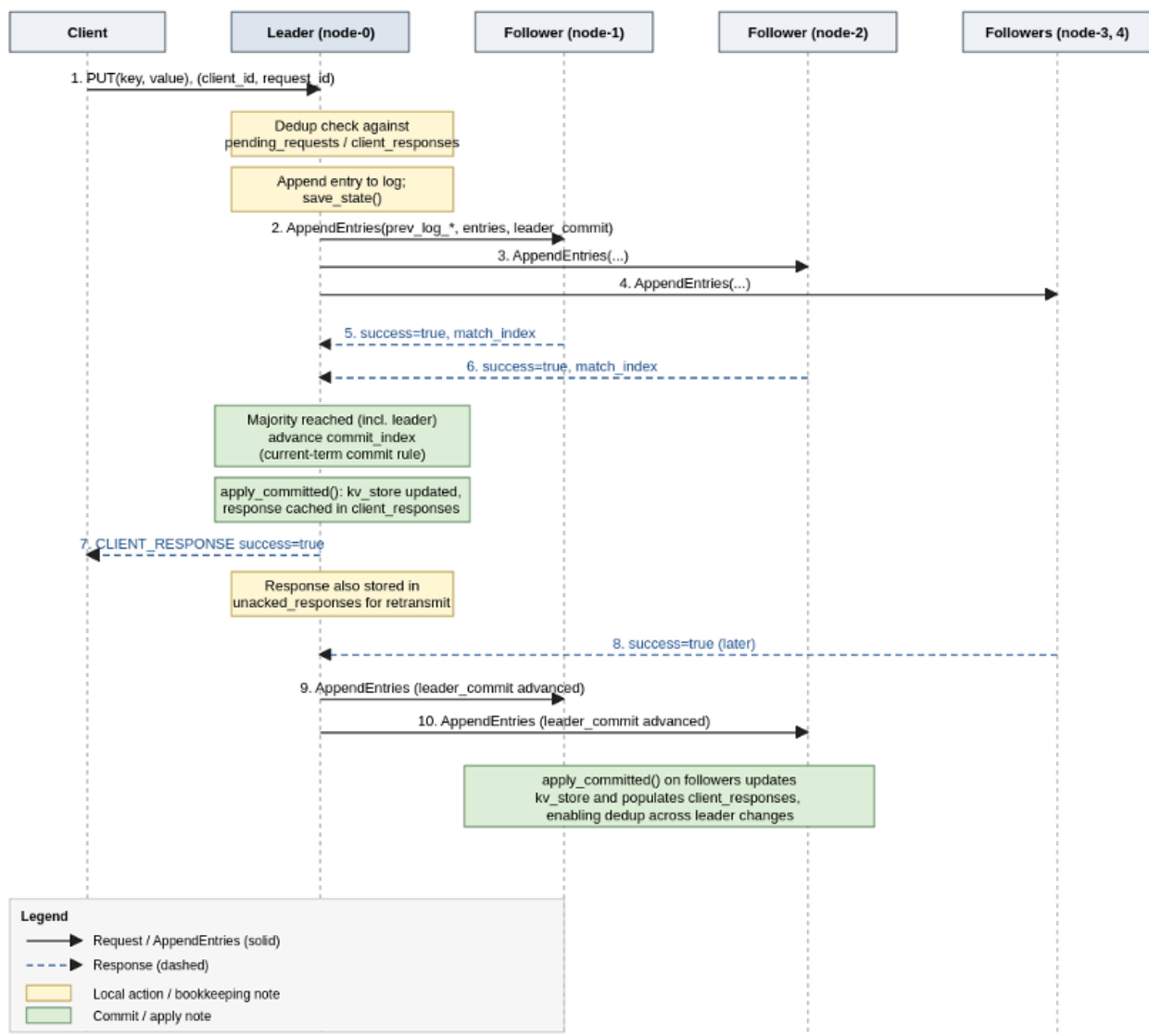


Diagram 3. End-to-end sequence for a client PUT or DELETE. The leader appends the entry, replicates via `AppendEntries`, advances `commit_index` once a majority has acknowledged, applies the entry to `kv_store` in `apply_committed()`, and only then returns a success response to the client. The response is also placed in `unacked_responses` so it can be retransmitted if dropped.

6. Client Request Handling and Deduplication

Client requests enter the cluster through `handle_client_request()`. The behaviour depends on the receiver's role and on whether the request has been seen before.

When a non-leader receives a client request, it forwards the message to the known `leader_id`. If no leader is known (for example during an election), it replies immediately with `success=false` and an error of "NOT LEADER". The forwarding path is preferred because it avoids two followers independently sending duplicate replies to the client; the leader is the single point that produces the canonical response.

On the leader side, the request key (`client_id`, `request_id`) is checked against three structures. If the key already appears in `pending_requests`, the request is in flight and is silently dropped (the existing log entry will produce the response). If the key appears in `client_responses`, a cached response is returned immediately, which is what makes the deduplication coherent across leader changes: `client_responses` is populated inside `apply_committed()`, so any node that has applied a request will know its outcome regardless of which node processed the original. Otherwise the request is new.

For PUT or DELETE the leader appends a log entry containing the index, term, operation, key, value, and the (`client_id`, `request_id`) pair. It then persists the log via `save_state()` and triggers a heartbeat round to start replication. The client only receives a success response after `apply_committed()` runs against the entry, that is, after a majority has accepted it. The

`unacked_responses` table holds recently sent responses with a timestamp; the response is resent on heartbeat ticks in case the original was dropped, until the client's eventual retry hits the cached-response path.

GET is handled differently and is described in Section 8.

It is worth being careful about the language here. The system prevents the most common form of duplicate application caused by client retries, and it does so reliably for the operations exposed by the API. It is not a fully exactly-once linearizable implementation in the textbook sense, because state such as `client_responses` is only kept on each node's in-memory map and is bounded by the same lifetime as the node process; a long-running deployment would also need a way to garbage-collect stale entries safely (see Section 11).

7. Persistence, Snapshotting, and Recovery

Raft requires three pieces of state to survive a crash: `current_term`, `voted_for`, and the log. Without persistent term and vote state, a restarted node could vote twice in the same term; without a persistent log, committed entries could be lost.

`save_state()` writes a JSON document containing `current_term`, `voted_for`, log and the snapshot dictionary to `data/<node-id>.json`. The write is staged into a tmp file and then renamed with `os.replace`, which on POSIX systems is atomic. This avoids leaving a half-written state file if the process crashes mid-write. `save_state()` is called every time a relevant field changes: on log append, on truncation, on vote grant, on step-down, on snapshot.

`load_state()` runs in the constructor. If a state file exists, `current_term`, `voted_for` and log are restored, and the snapshot is loaded through `load_snapshot()`, which seeds `kv_store`, `last_applied` and `commit_index` from the snapshot's `last_included_index` and `last_included_term`. The volatile pieces (role, leader_id, election timers, deduplication maps) start fresh. This is the standard Raft restart contract: the node re-enters the cluster as a follower, learns of the current leader through the next `AppendEntries`, and catches up via normal replication.

7.1 Snapshotting

An unbounded log is impractical, so `apply_committed()` triggers `take_snapshot()` whenever the log length exceeds `SNAPSHOT_THRESHOLD` (100 entries in `config.py`). The snapshot captures `last_included_index`, `last_included_term` and a copy of `kv_store`, and the log is truncated to keep only entries with index strictly greater than `last_included_index`. The snapshot is stored alongside the log in the same state file, so `save_state()` persists both atomically.

One important limitation: there is no `InstallSnapshot` RPC. If a follower falls so far behind that the leader has already snapshotted past the entries that follower needs, the leader cannot transmit the snapshot to bring the follower up to date. In normal operation on a five-node cluster this case is unlikely, because the snapshot threshold is well above the gap that ordinary backtracking can repair, but the limitation is real and is recorded in Section 11.

8. Behaviour Under Failures

The supplied network variants exercise three distinct failure modes: variable delays, message drops, and partitions. Raft's correctness arguments hold under all three, but the visible behaviour differs.

8.1 Delayed messages and overlapping elections

Delays alone do not change the protocol's correctness. They can, however, cause two followers to time out at roughly the same moment, both run elections, and split the vote. When this happens, neither candidate reaches a majority. Both fall back to the follower role on the next round and the randomised election timeout makes it unlikely that they collide again on the next attempt. The term ordering does the rest: any message carrying a strictly higher term causes the receiver to step down, so a candidate who wins ends the dispute as soon as its first heartbeat arrives at the loser.

8.2 Message loss

Lost `AppendEntries` are repaired by the next heartbeat round, because the leader resends from `next_index` every tick. Lost responses are repaired by the same mechanism on the leader side: a follower whose response was dropped will simply receive another `AppendEntries` and respond again. Lost client responses are handled separately through `unacked_responses`: recently sent responses are retained temporarily and resent on heartbeat ticks in case the original was dropped, until the client retries with the same `request_id` and triggers the cached-response path. Backtracking via the `match_index` hint then closes any gap that has built up.

8.3 Network partitions

A partition splits the cluster into a majority side and a minority side. The partition variant in the test harness alternates ten seconds of partition with ten seconds of healing, splitting the five-node cluster two-against-three. The majority side can continue normally: it has at least three nodes, which is enough to elect a leader and commit new entries. The minority side has at most two nodes and cannot form a quorum on its own, so it cannot commit anything and cannot elect a new leader. A node that was leader before the partition and is now on the minority side may continue to think of itself as leader for a short window, until it tries to satisfy a request and discovers it cannot reach a majority. This is what is meant by a stale leader.

The system favours consistency over availability in this case, which matches the Raft design and the well-known CAP trade-off (Gilbert and Lynch, 2002; Terry et al., 1995). Clients on the minority side see timeouts rather than incorrect responses. When the partition heals, the side with the higher term takes over, or, with Pre-Vote enabled, the standing leader stays in office, see Section 9, and replication catches up the previously isolated nodes.

8.4 Heartbeat-confirmed (ReadIndex-style) reads

GET is the operation most exposed to partition-related staleness. A naive read served from the leader's local `kv_store` can return a value that has since been overwritten by a new leader on the other side of a partition. To avoid this, GET is handled with a ReadIndex-style mechanism.

When the leader receives a GET, it allocates a fresh `read_id`, records the read in `pending_reads` with itself as the only initial acknowledger, and broadcasts a heartbeat tagged with that `read_id`. Followers attach the same `read_id` to their `AppendEntries` response. The leader marks each acknowledger in `pending_reads[read_id]["acks"]`, and only when a majority of acks have arrived does it read from `kv_store` and reply to the client. The reply is also placed in `unacked_responses` so that a dropped reply is retried.

This is described as ReadIndex-style rather than full ReadIndex (Ongaro, 2014) because the implementation does not block reads behind a no-op append at the start of a leader's term. In Ongaro's full description, the new leader commits one no-op entry from its current term before answering any reads, which establishes that any value committed by a previous leader is visible. Without that step, a freshly elected leader could in principle answer a read using state from before the entries it has not yet finished applying. In practice `apply_committed()` runs eagerly, but the absence of the no-op step is the reason the report uses cautious wording: the mechanism improves read safety by preventing answers from a leader that cannot confirm recent majority contact, but it is not a full production-grade ReadIndex.

If quorum cannot be assembled, the read remains in `pending_reads`. `send_heartbeats()` retries each pending read after two heartbeat intervals by re-broadcasting tagged heartbeats and resetting the ack set. After roughly ten heartbeat intervals the read is abandoned and a `CLIENT_RESPONSE` with `error="READ TIMEOUT"` is sent so the client can retry without waiting on its full socket timeout.

9. Extension: Pre-Vote and Robust Client Handling

9.1 Introduction and motivation

The base Raft implementation submitted earlier in this coursework provides leader election, log replication of PUT and DELETE operations, persistent term and log state, commit-driven application of entries to a key-value state machine, and the standard request/response cycle for PUT, GET and DELETE. The cluster runs five nodes (node-0 to node-4) with an election timeout of 1.5 to 3.0 seconds and a 0.5 second heartbeat interval, as set in `config.py`.

On a clean network this works as expected. However, basic Raft has a known weakness under unreliable networks: when a follower stops receiving heartbeats it immediately increments `current_term`, becomes a CANDIDATE, and broadcasts `RequestVote`. If that follower is partitioned away from the majority, the election fails, the timer fires again, and the term keeps climbing. When the partition heals, the rejoining node carries a much higher term than the rest of the cluster, which forces the healthy leader to step down. Safety is preserved, but availability suffers because the cluster runs an unnecessary election.

The main extension implemented here is the Pre-Vote phase from Ongaro's thesis. Before incrementing its term, a follower first asks the cluster whether a majority would grant it a vote in a hypothetical next term. Only if a majority say yes does the node move to CANDIDATE and increment `current_term`. An isolated node is therefore prevented from

inflating its term while it is in the minority partition, since peers on the majority side refuse the probe as long as their leader is still alive.

Alongside Pre-Vote, the submitted implementation includes supporting features that improve client-visible behaviour under partitions and message loss: deduplication of client requests via (client_id, request_id) keys, retransmission of unacknowledged responses, follower-to-leader forwarding, heartbeat-confirmed reads, and stale-leader awareness via abandonment of pending reads when a leader can no longer reach a majority. Diagram 2 contrasts the two election flows.

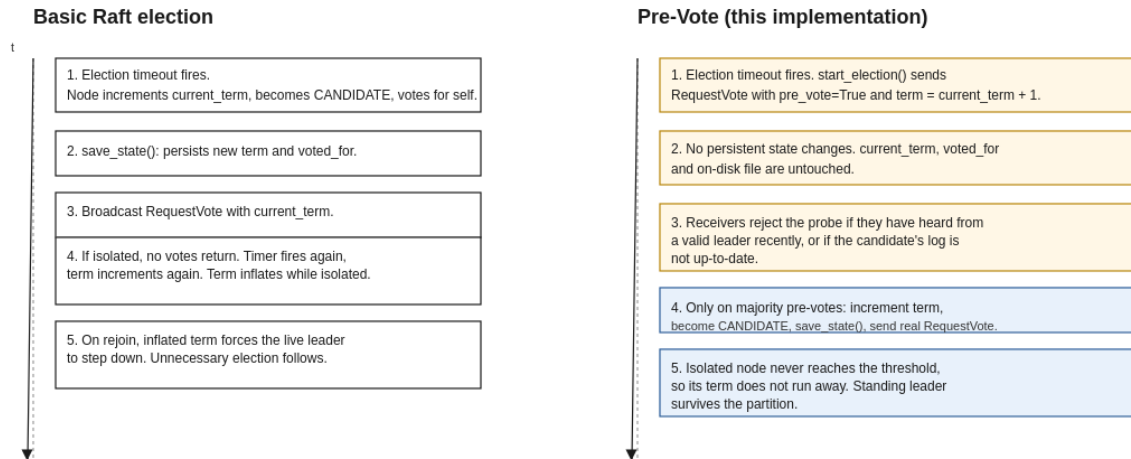


Diagram 41. Basic Raft election flow vs. Pre-Vote election flow.

9.2 Problem scenario: partition and term inflation

Consider the five-node cluster used here. Suppose node-0 is the elected leader and the cluster is operating normally. The network partition simulator (network_partition.py) then splits the cluster so that node-0, node-1, node-2 and node-4 remain on one side and node-3 is isolated alone. The majority side has four out of five nodes (above the quorum of three) and can continue to elect leaders and commit entries. The minority side has one node and can never form a quorum on its own.

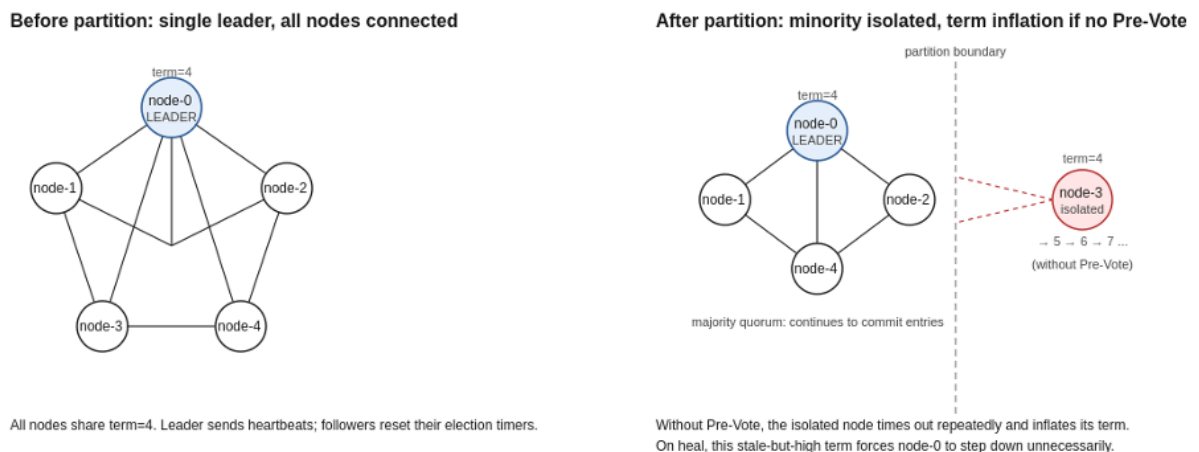


Diagram 5. Five-node cluster before partition (left) and term inflation on the minority side after partition (right).

In standard Raft, node-3 misbehaves in this scenario. It stops receiving heartbeats, times out within at most three seconds, increments current_term, becomes a CANDIDATE and broadcasts RequestVote. The messages cannot reach the majority side, so the election fails, the timer fires again, current_term increases, another election fails, and so on. By the time the partition heals after ten seconds (PARTITION_DURATION in config.py), node-3 may be at term 8 or higher. When connectivity is restored, the next AppendEntries from node-0 carries the original term 4. node-3 replies with its higher term, and the standard step-down rule then forces node-0, a perfectly healthy leader, to step down. The cluster runs another election before settling. None of this would have been needed if node-3 had simply rejoined as a follower.

This is not a safety violation. Raft never commits conflicting entries here, and no client sees a wrong value. The cost is purely availability: the cluster is unavailable during the unnecessary election, and any client requests overlapping that

window will time out. In the chaos and partition tests this shows up as occasional client retry storms after a partition heals.

9.3 Design and implementation of Pre-Vote

The Pre-Vote phase is implemented in `node.py`. It re-uses the existing `REQUEST_VOTE` message type rather than adding a new RPC, with a new boolean field `pre_vote` attached to both the request and response. This keeps the wire format minimal while clearly separating probes from real votes.

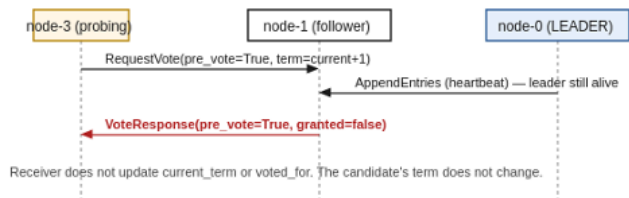
Probe round. When the election timer fires, control passes from `_timer_loop` to `start_election()`. Instead of incrementing the term, the node sets `pre_vote_term` to `current_term + 1`, initialises `pre_votes_received` to its own id, and broadcasts a `REQUEST_VOTE` with `pre_vote=True` and `term=current_term+1`. `current_term`, `voted_for` and the persistent state file on disk are all left unchanged.

Receiver decision. On receipt, `handle_request_vote()` inspects the `pre_vote` flag and takes a dedicated probe path. A pre-vote is granted only when (i) the hypothetical term is at least the receiver's `current_term`, (ii) the receiver has not heard from a valid leader recently, and (iii) the candidate's log is at least as up-to-date as the receiver's under the standard (`last_log_term`, `last_log_index`) rule. The leader-alive check in (ii) is what makes Pre-Vote effective: a follower that is still receiving heartbeats refuses to support the probe at all. Whatever the decision, the receiver does not modify any persistent field, does not record `voted_for`, does not raise `current_term`, and does not call `save_state()`. It simply replies with a response that mirrors `pre_vote=True`.

Tally and promotion. Pre-vote responses are handled in `handle_request_vote_response()`. Responses whose term does not match `pre_vote_term` are ignored, which avoids miscounting stale replies. If a response carries a strictly higher term, the node steps down and clears its pre-vote state, the same defensive rule used in standard Raft. Granted pre-votes are accumulated in `pre_votes_received`, and only when this set reaches a majority of `CLUSTER_SIZE` (three out of five) does the node commit to a real election: it increments `current_term`, sets role to `CANDIDATE`, votes for itself, persists the new state, and broadcasts a fresh `RequestVote` with `pre_vote=False`.

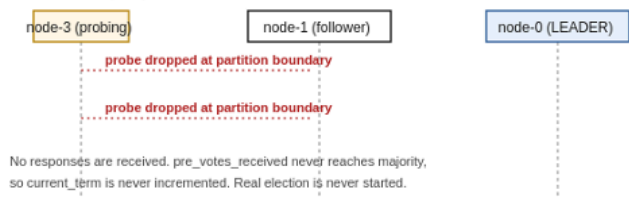
Case A: probes reach majority, all rejected

Candidate is still receiving some traffic from peers, but those peers still hear from a live leader.



Case B: fully isolated, no responses arrive

Candidate is completely cut off from the rest of the cluster.



Pre-Vote rules implemented in `node.py`

`start_election()`:

- `pre_vote_term = current_term + 1`
- `pre_votes_received = {self.node_id}`
- broadcast `RequestVote(pre_vote=True)`

`handle_request_vote()` (probe path):

- reject if hypothetical term < `current_term`
- reject if a leader has been heard from recently
- otherwise apply log up-to-date check
- never modify `current_term`, `voted_for`, or disk

`handle_request_vote_response()` (probe path):

- step down on strictly higher term
- ignore stale responses (`term != pre_vote_term`)
- count granted probes in `pre_votes_received`
- on majority: increment term, become `CANDIDATE`, `save_state()`, send real `RequestVote`

Outcome: an isolated node cannot inflate its term.

Live leaders survive partitions without disruption.

Diagram 6. Pre-Vote message sequence for node-3 timing out while the majority side still recognises an active leader.

The effect is similar in both partial-delay and full-partition cases. If node-3 can still contact the majority side, its pre-vote probes are rejected because those nodes have recently heard from the current leader. If node-3 is fully isolated, its probes receive no responses at all. In both cases it never reaches the majority threshold, so it does not increment `current_term`. When the partition heals, its term has not run ahead of the rest of the cluster, so the standing leader is not forced to step down.

9.4 Supporting robustness features

Pre-Vote alone improves stability under partitions, but a robust client-facing system also has to deal with retried requests, leader changes, and stale leaders that are still issuing replies. Three additional features address this.

9.4.1 Duplicate client request handling

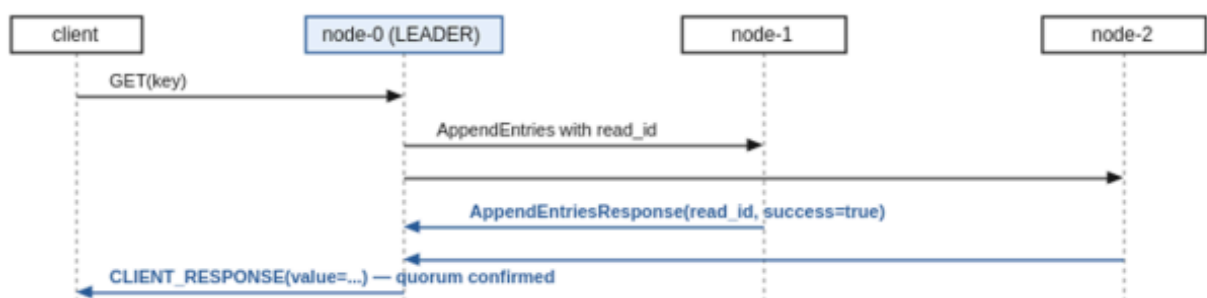
The same client request can arrive at the cluster more than once: a PUT or DELETE may have been applied on the leader, but the response was lost or the leader changed before sending it, and the client's socket times out and retries. Without protection, the cluster would apply the operation twice, which is unsafe for non-idempotent operations. Each request carries a `request_id`, and the node tags it with the originating `client_id`, giving a request key. Three structures are kept on every node: `pending_requests` (in-flight keys, dropped on duplicate before any work is done), `client_responses` (a cache mapping completed keys to their responses, replayed on retry), and `unacked_responses` (a small map in which recently sent responses are retained with timestamps and resent on heartbeat ticks in case the original response was dropped). These structures are populated on both leader and followers when a committed entry is applied in `apply_committed()`, so after a leader change the new leader can still recognise a duplicate that the old leader had begun handling, because the request appears in the committed log.

9.4.2 Follower forwarding and stale-leader handling

A client does not necessarily know which node is the leader, and `client_test.py` simply broadcasts each request to every node. In `handle_client_request()`, a non-leader does not process the request itself: if it knows `leader_id` it forwards the request to that leader, otherwise it returns NOT LEADER and the client retries. The leader is then responsible for completing successful requests, which keeps the cached-response logic coherent.

GET requests use heartbeat confirmation rather than being answered locally from `kv_store`. A naive local read is unsafe under partitions, since a deposed leader on the minority side may not yet know it has been deposed, and could return a value that has since been overwritten by the new leader. When the leader receives a GET it allocates a fresh `read_id`, records the read in `pending_reads` with itself as the only initial acknowledger, and broadcasts a heartbeat tagged with that `read_id`. Followers reply as part of their normal `AppendEntries` response, and the leader returns the value to the client only when a majority of acks (including itself) have arrived. A pending read is retried on each heartbeat tick, and is abandoned with a `READ TIMEOUT` after roughly ten heartbeat intervals (about five seconds). This abandonment is what implements stale-leader awareness: a leader that has lost contact with the majority cannot assemble a quorum for any pending GET, so its reads time out instead of returning a possibly stale value. The leader will eventually step down on seeing a higher term, but the key point is that no read is satisfied from a leader that cannot confirm recent majority contact.

Majority-side leader: heartbeat-confirmed GET succeeds



Minority-side stale leader: cannot form a read quorum

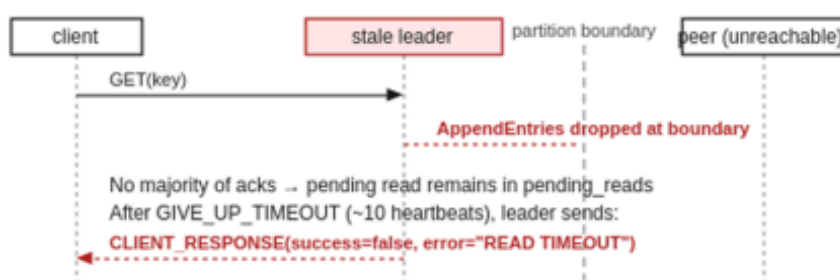


Diagram 7. Behaviour during a partition for a GET request: the majority leader confirms quorum and replies, while a stale minority leader cannot form a read quorum and times out instead of returning a stale value.

10 Testing and Evaluation

Testing used the supplied `client_test.py` against the same five-node cluster, under each of the network modes provided: clean, delayed (50 to 500 ms latency), lossy (10% drop rate plus delays), partition (alternating ten-second partitions and ten-second heals), and chaos (the most hostile combination of all of the above with a doubled drop rate). The behaviours observed across these runs are summarised in Table 2.

Network mode	Expected behaviour	Observation
Clean	Leader is elected within a few seconds; PUT, GET and DELETE complete consistently. No timeouts.	Tests passed consistently across observed runs. The leader stabilises after the first election and successive operations complete on the first attempt.
Delayed	Reordered messages may briefly cause split votes; protocol converges. Operations complete after at most a small number of retries.	Tests passed in observed runs. Occasional retries are observed; the final state matches expectations.
Lossy	Dropped AppendEntries are repaired by the next heartbeat. Dropped client responses are recovered through <code>unacked_responses</code> or via the <code>client_responses</code> cache on retry.	Tests passed in observed runs. Throughput is lower than on the clean network because of replication retries.
Partition	Majority side commits and serves requests; minority side cannot. Leader stays consistent across heals when Pre-Vote is enabled.	Tests passed in observed runs. After each heal a single leader is restored and replication catches up the previously isolated nodes.
Chaos	Combined delays, doubled drop rate and randomised partitions; some operations expected to time out.	Tests are non-deterministic and do not always pass. Failures observed are availability failures (timeouts), not correctness failures.

Table 2. Test results summary across the supplied network variants.

It is worth distinguishing two kinds of failure. A correctness failure occurs when a confirmed write is later observed as lost or overwritten with a different value, or when a GET returns a value no client ever PUT. An availability failure occurs when a request times out or returns NOT LEADER and the client has to retry. Raft is intended to eliminate the first class entirely and to bound the second by the duration of a leader change.

The chaos network is the only mode where failures were observed at the test level, and the failures observed in those runs were of the second kind: occasional timeouts and READ TIMEOUT responses during active partitions or just after a leader change. This matches the assignment guidance, which is that chaos tests are non-deterministic and need not always pass; what matters is that no confirmed write was followed by a GET returning a different value. No such instance was observed.

11. Limitations and Trade-offs

A few limitations of the implementation are worth recording explicitly.

First, the system favours consistency over availability during minority partitions. A client whose request is routed to the minority side will see a timeout rather than a stale value, which is the correct behaviour for Raft but is visible as failed requests in the test output.

Second, heartbeat-confirmed reads add latency. Every GET incurs at least one heartbeat round-trip to the followers before the client gets a response. In return, the leader cannot answer a read it cannot confirm. The mechanism is described as ReadIndex-style rather than full ReadIndex because the implementation does not commit a no-op entry at

the start of a new leader's term, so a freshly elected leader could in principle answer a read using state that is correct but not yet committed in the strictest sense. Section 8 discusses this in more detail.

Third, snapshotting is implemented locally but the `InstallSnapshot` RPC is not. If a follower falls so far behind that the leader has already truncated the entries the follower needs, the leader cannot transmit the snapshot to bring the follower up to date. The snapshot threshold is high enough that this is unlikely in normal operation, but the limitation is real.

Fourth, the chaos network is non-deterministic by design. The combination of a doubled drop rate, variable delays and randomised partitions means that timing-sensitive code paths can fail to make progress within the test timeout. The lecturer's announcement explicitly acknowledges this. The implementation aims to ensure that timeouts under chaos are availability failures and not correctness failures.

Finally, this is a coursework-scale implementation. Production Raft systems, and closely related primary-backup protocols such as ZooKeeper's Zab (Junqueira, Reed and Serafini, 2011), include a configuration-change protocol (joint consensus or single-server changes), proper `InstallSnapshot`, lease-based reads or full `ReadIndex` with a no-op-on-leader-change step, batched and pipelined replication, and exactly-once client semantics that survive client identity rotation. None of those is in scope for the assignment, and none is implemented here.

Conclusion

This project implemented a five-node replicated key-value store on top of Raft. The base implementation covers leader election with the standard term, vote and log-freshness rules; log replication with a `prev_log_index/prev_log_term` consistency check and `match_index`-based backtracking; majority-based commitment that respects the current-term commit rule; and in-order application of committed entries to the key-value store. Persistence stores `current_term`, `voted_for`, log and snapshot atomically, and snapshotting truncates the log past `last_applied` so that the log does not grow without bound.

On top of that, the implementation handles client requests with a deduplication scheme keyed on `(client_id, request_id)`, forwards client requests from followers to the known leader, and serves GET only after a heartbeat-confirmed majority round so that a leader which has lost contact with the majority cannot return a value. The Pre-Vote extension adds a probe round before any term increment, which removes the term-inflation behaviour of basic Raft when a node is isolated by a partition. This is an availability improvement; it does not change Raft's safety model, and the report has been deliberately careful not to claim it does.

Across the supplied clean, delayed, lossy and partition networks, the cluster is expected to pass the assignment tests consistently and did so in observed runs. On the chaos network the tests are non-deterministic by design, and occasional availability failures are observed, which matches the guidance that chaos runs need not always pass. No correctness failures, in the sense of a confirmed write later returning a conflicting value, were observed in any run.

References

- [1] D. Ongaro and J. Ousterhout, "In Search of an Understandable Consensus Algorithm", USENIX Annual Technical Conference, 2014.
- [2] D. Ongaro, "Consensus: Bridging Theory and Practice", PhD thesis, Stanford University, 2014.
- [3] A. S. Tanenbaum and M. Van Steen, Distributed Systems: Principles and Paradigms, Prentice Hall.
- [4] M. J. Fischer, N. A. Lynch and M. S. Paterson, "Impossibility of Distributed Consensus with One Faulty Process", Journal of the ACM, vol. 32, no. 2, pp. 374–382, 1985.
- [5] S. Gilbert and N. Lynch, "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services", ACM SIGACT News, vol. 33, no. 2, pp. 51–59, 2002
- [6] D. Ongaro and J. Ousterhout, "The Raft Consensus Algorithm", raft.github.io. [Online]. Available: <https://raft.github.io/>
- [7] L. Lamport, "The Part-Time Parliament", ACM Transactions on Computer Systems, vol. 16, no. 2, pp. 133–169, May 1998.
- [8] F. P. Junqueira, B. C. Reed and M. Serafini, "Zab: High-performance broadcast for primary-backup systems", in Proc. 41st IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), 2011, pp. 245–256.
- [9] D. B. Terry, M. M. Theimer, K. Petersen, A. J. Demers, M. J. Spreitzer and C. H. Hauser, "Managing update conflicts in Bayou, a weakly connected replicated storage system", in Proc. 15th ACM Symposium on Operating Systems Principles (SOSP), 1995, pp. 172–183.